

ConfFit — Agentic System for Academic Conference Submission

Architecture Design Document

Course Project | Architecture: a lightweight **Supervisor Agent** coordinating two worker agents — a **Reflection Agent** (framing) and a **ReAct Agent** (format). Deliberately kept small. An author-facing agent that adapts a paper to a target conference and fixes its format.

1. Problem & Scope

Researchers re-submit the same core work to different venues. Each venue has (a) a different **focus** (representation learning vs. applied/systems vs. theory) and (b) different **formatting and submission rules** (template, citation style, page limits, double-blind anonymity). Doing this by hand for every venue is slow and error-prone.

ConfFit takes a manuscript plus a target conference and delivers two things:

1. **Framing** — how to re-position and re-emphasize the paper so its contribution lands with *this* venue's reviewers, grounded in the conference's Call-for-Papers (CFP) and the emphasis of past accepted papers.
2. **Format** — a compliance check against the venue's submission rules (template, citation style, page/section limits, anonymity/double-blind) that returns **both** a suggestions report *and* a corrected ("fixed") version of the manuscript.

Citation verification is intentionally **out of scope** to keep the project small and cheap.

2. Architecture decision

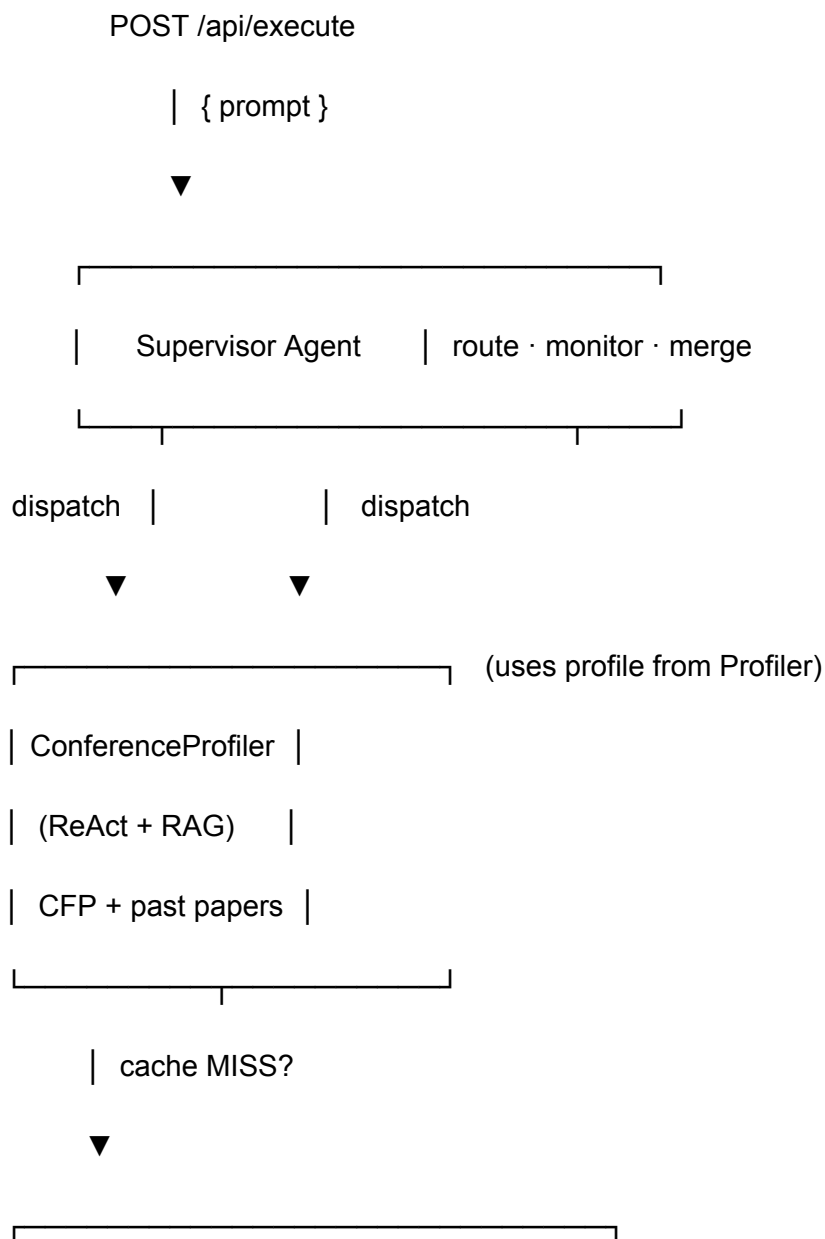
The problem has two loosely-coupled tasks that both depend on knowing the target venue, so the design is a **Supervisor Agent** (top-level coordinator) over two specialized workers. This keeps it a genuine agent system — the Supervisor decides at runtime which workers to run — without the weight of a plan/replan engine or a multi-tool monolith.

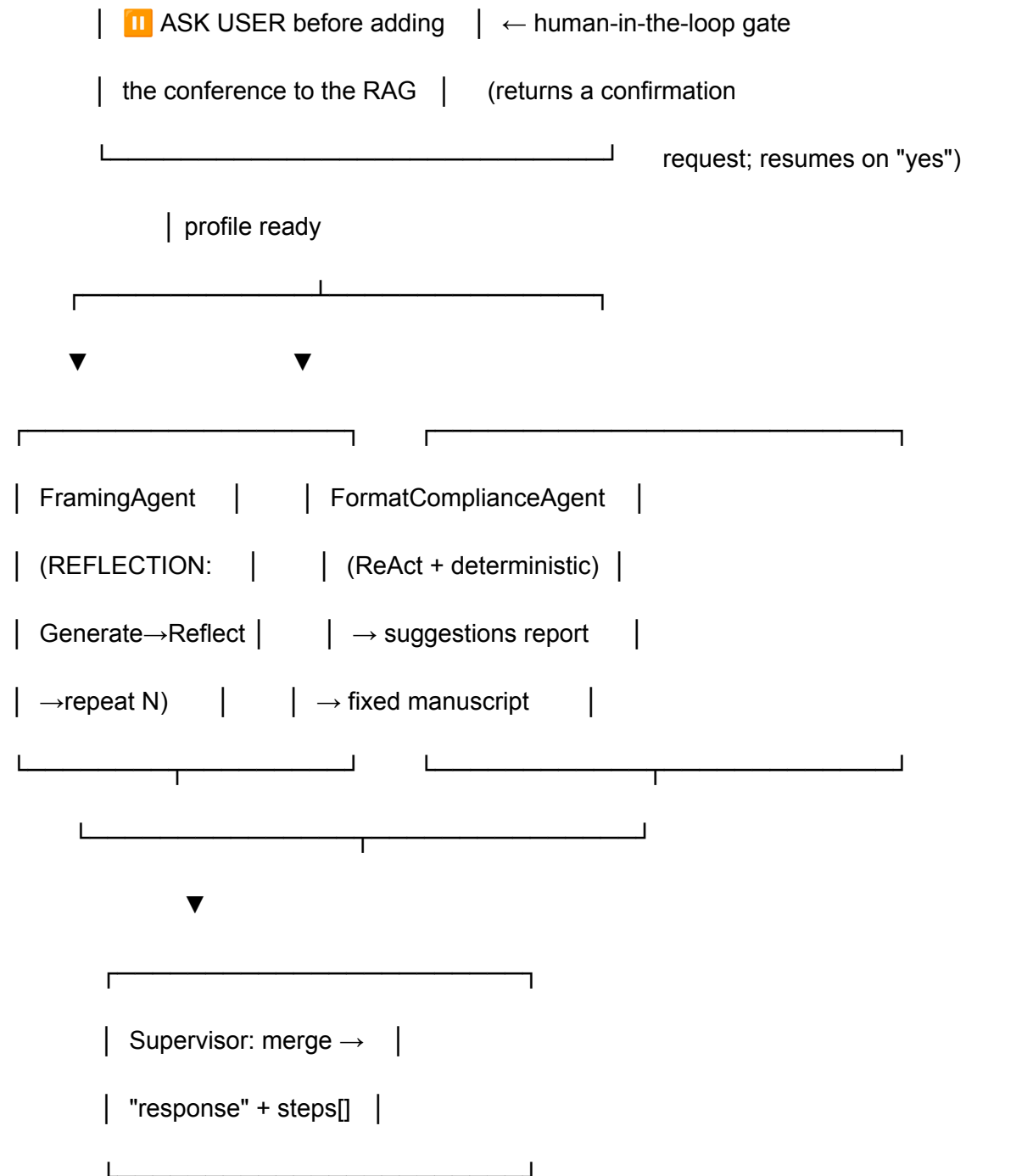
The two workers use the patterns that fit each task best:

- **Framing = Reflection Agent** (Lecture 7). Repositioning a paper is a writing / judgement task with no single right answer — exactly what Reflection's Generate → Reflect → repeat loop is "good for."
- **Format = ReAct Agent** (Lecture 5/6). Rule checking is mostly deterministic; the agent reasons and calls a lookup tool only when a rule is unclear.

The Supervisor is intentionally thin: one routing decision at the start and one merge at the end.

3. Topology





Shared services (not agents):

Supabase → conference-profile cache, run/memory store

Pinecone → embeddings of CFP chunks + past accepted papers (text-embedding-3-small)

MCP Tools → web_search, web_fetch, vector_search, rules_lookup

Every agent box maps 1:1 to a `module` name in the `steps` trace.

4. Agents

4.1 Supervisor Agent (thin coordinator)

On each request it: (1) reads the `prompt`, identifies the target conference and whether the user wants framing, format, or both (one LLM call — this runtime choice is what makes it an agent, not a fixed pipeline); (2) invokes `ConferenceProfiler` first, since both workers need the profile; (3) dispatches the relevant worker(s); (4) merges their outputs into the final `response`.

Deterministic pre-processing (splitting the manuscript into sections) runs in code — no tokens spent on parsing. Logged as module `Supervisor`.

4.2 ConferenceProfiler Agent (ReAct + RAG, with human-in-the-loop)

Job: produce a structured **Conference Profile** — scope/tracks, what the venue rewards, the emphasis of recently accepted papers, and the raw submission rules.

1. **Cache check (Supabase / Pinecone, no LLM):** if the venue's profile already exists, return it — the biggest cross-run saving.
2. **On cache MISS → ask the user first.** The agent does **not** silently crawl and ingest. It returns a confirmation request through the Supervisor: *"I don't have guidelines for <conference> yet. May I fetch <proposed CFP URL> and add it to the knowledge base? Reply yes, or paste the correct link."* Nothing is written to the RAG until the user approves.
3. **After approval:** a short ReAct loop uses `web_fetch/web_search` to pull the CFP + accepted-papers list, embeds the chunks into Pinecone, synthesizes the profile, and persists it to Supabase (so the next run is a cache hit).

Output: `ConferenceProfile { focus_areas, valued_criteria, accepted_paper_emphasis, format_rules }`. Logged as `ConferenceProfiler`.

4.3 FramingAgent (Reflection Agent)

Given `paper_contributions` + `ConferenceProfile`:

1. **Generate** (LLM, few-shot): a repositioning proposal — suggested angle, which contributions to foreground/background, and rewritten title/abstract/intro-opening variants aligned to the venue's values.
2. **Reflect** (LLM): critique against the profile (Critique / Pros / Cons) — "does this match what the venue rewards? is any claim unsupported by the paper?"
3. **Repeat N times** (N capped at 1–2 to avoid over-reflecting and to protect the budget).

Output: framing recommendations + refined abstract/intro drafts. Logged as **FramingAgent** (generate) and **FramingReflect** (critique).

4.4 FormatComplianceAgent (ReAct) — report **and** fixed version

Job: check the manuscript against the venue's rules and return **two artifacts**.

- **Checks** are deterministic where the rule is known (page/section limits; citation style e.g. IEEE/ACM/APA; anonymity scan for author names, affiliations, "our prior work [X]", de-anonymizing repo links) — no LLM cost. A short ReAct loop calls **rules_lookup** only when a rule is ambiguous.
- **Artifact 1 — suggestions report:** a checklist { **rule**, **status**: pass/fail/warn, **suggestion** }.
- **Artifact 2 — fixed manuscript:** the agent applies the fixes. Mechanical fixes (citation-style conversion, redacting author/affiliation blocks for double-blind, stripping acknowledgements) are done in code; fixes needing judgement (e.g., trimming an over-length abstract) use **one** LLM call. It returns the corrected text alongside the report.

Logged as **FormatComplianceAgent**.

5. Data & tool layer

Layer	Technology	Use
Text model	MB5R2CF-azure/gpt-5.4-mini	agent reasoning/generation
Embeddings	MB5R2CF-azure/text-embedding-3-small	CFP + past-paper chunks
Primary DB	Supabase	conference-profile cache, run history, conversation memory

Layer	Technology	Use
Vector DB	Pinecone	RAG retrieval (namespaced per conference)
Tools (MCP)	<code>web_search</code> , <code>web_fetch</code> , <code>vector_search</code> , <code>rules_lookup</code>	exposed via <code>tools/list</code> / <code>tools/call</code> (Lecture 7 MCP)

6. Cost, latency & simplicity (maps to Requirement 1)

Kept small on purpose. The Supervisor **routes**, so a "format only" request skips the Framing agent. Conference profiles are **cached** in Supabase (the expensive RAG build happens once per venue, and only after the user approves it). Section splitting, format/anonymity checks, and mechanical fixes run **in code**, not in the LLM. The Reflection loop is capped ($N \leq 2$). Each worker receives **only its slice** of context. A full run is roughly **6–9 small LLM calls** — comfortably inside the \$13 budget and the 300 s Vercel ceiling.

7. Mapping to required API endpoints

- **GET /api/team_info** — static team JSON.
- **GET /api/agent_info** — `description`, `purpose`, `prompt_template`, `prompt_examples` with full `steps`.
- **GET /api/model_architecture** — PNG of the §3 topology; every box name matches the `steps` module names.
- **POST /api/execute** — returns `{ status, error, response, steps }`.
`module` $\in$ {Supervisor, ConferenceProfiler, FramingAgent, FramingReflect, FormatComplianceAgent}.

Human-in-the-loop response shape: when the conference is missing from the RAG, `/api/execute` returns a normal `status:"ok"` whose `response` is the confirmation question (and the trace shows `ConferenceProfiler` reporting a cache miss). The user's next prompt ("**yes**" or a link) resumes ingestion. This uses the optional follow-up-prompt support in the GUI — no extra endpoint needed.

8. Example **steps** traces (abridged)

8a. Cache miss → asks the user (no ingestion yet)

```
[  
  
  { "module": "Supervisor",  
  
    "prompt": { "system_prompt": "Decide target conference and which workers to run; return  
JSON...",  
  
      "user_prompt": "Format-check my paper for ICLR 2027: <manuscript>" },  
  
    "response": { "target": "ICLR 2027", "dispatch":  
["ConferenceProfiler", "FormatComplianceAgent"] } },  
  
  { "module": "ConferenceProfiler",  
  
    "prompt": { "system_prompt": "Check the cache. If missing, DO NOT ingest — return a  
confirmation request...",  
  
      "user_prompt": "Profile ICLR 2027." },  
  
    "response": { "cache_hit": false,  
  
      "ask_user": "I don't have ICLR 2027 guidelines yet. May I fetch  
https://iclr.cc/.../CallForPapers and add it to the knowledge base? Reply yes or paste the correct  
link." } }  
]
```

response returned to the user = the **ask_user** question. No writes to Pinecone/Supabase.

8b. User replies "yes" → ingest, then format

```
[  
  
  { "module": "ConferenceProfiler",  
  
    "prompt": { "system_prompt": "User approved. ReAct: fetch the CFP, embed, build the  
profile...",
```

```

      "user_prompt": "yes" },

      "response": { "focus_areas": ["representation learning"], "format_rules": {
"style": "ICLR/natbib", "page_limit": 9, "anonymous": true } } },

      { "module": "FormatComplianceAgent",

        "prompt": { "system_prompt": "Produce (1) a suggestions checklist and (2) the fixed
manuscript...",

          "user_prompt": "{manuscript:<...>, format_rules:{...}}" },

          "response": { "report": [ { "rule": "anonymity", "status": "fail", "suggestion": "remove author block
on p.1" },

            { "rule": "citation_style", "status": "warn", "suggestion": "convert 3 refs to natbib"
} ],

            "fixed_manuscript": "<corrected text with author block redacted and refs
reformatted>" } },

          { "module": "Supervisor",

            "prompt": { "system_prompt": "Merge into a readable response...", "user_prompt": "{report:[...],
fixed:<...>}" },

            "response": { "response": "2 issues found and fixed. Report + corrected manuscript below." } }

    ]

```

9. Suggested **prompt_template** (for `/api/agent_info`)

Target conference: <name or CFP URL, e.g. "ICLR 2027">

Task: <framing | format | both>

Paper: <paste manuscript text, or abstract + contributions + bibliography>

Notes: <optional, e.g. "we were rejected from NeurIPS for being too applied">

10. Build order (suggested)

1. Endpoints + minimal GUI on Vercel; static `team_info` / `agent_info`; empty `execute`.
2. **Supervisor** with a stubbed worker — prove runtime routing and the `steps` trace.
3. **ConferenceProfiler** with the cache + human-in-the-loop confirmation gate (RAG on Pinecone/Supabase).
4. **FormatComplianceAgent** — report + fixed manuscript (mechanical fixes in code, one LLM call for judgement fixes).
5. **FramingAgent** — Reflection loop.
6. Supervisor merge + polish; render the architecture PNG; wire every agent into `steps`.